

Objetivo geral:

Familiarizar o aluno com a linguagem TypeScript por meio da reescrita progressiva de funções inicialmente desenvolvidas em JavaScript, promovendo a compreensão da tipagem estática, da modelagem de dados com interfaces, da manipulação de estruturas como arrays e objetos, e da organização de projetos utilizando ferramentas do ecossistema Node.js.

Objetivos específicos:

- Compreender o funcionamento da tipagem estática em TypeScript;
- Trabalhar com tipos primitivos e arrays tipados;
- Aplicar a técnica de tipagem por união (Union Types) em parâmetros e propriedades;
- Utilizar o operador `typeof` para realizar refinamento de tipos em tempo de execução;
- Declarar e utilizar interfaces para tipar objetos simples e compostos;
- Manipular listas de objetos com base em critérios condicionais;
- Reforçar boas práticas de declaração de funções com parâmetros e retorno tipados;
- Organizar projetos TypeScript com `ts-node`, `tsconfig.json` e scripts no `package.json`.

Tipagem estática:

Em linguagens de programação com **tipagem estática**, como TypeScript, o tipo de cada variável, parâmetro ou valor de retorno é definido de forma explícita (ou inferido) **antes da execução do programa**, geralmente no momento da escrita do código. Isso permite que muitos erros sejam detectados **em tempo de compilação**, promovendo maior segurança, clareza e previsibilidade no desenvolvimento.

Ao contrário da **tipagem dinâmica**, típica do JavaScript, onde os tipos podem mudar em tempo de execução, o TypeScript exige (ou infere) os tipos para garantir consistência. Essa abordagem reduz falhas comuns como:

- uso inadequado de variáveis;
- retornos inesperados de funções;
- atribuições incorretas;
- acesso indevido a propriedades de objetos.

O TypeScript oferece ainda recursos como **união de tipos** (`type1 | type2`), **refinamento com `typeof`**, **interfaces para modelagem de objetos**, e **funções genéricas**, tornando-se uma ferramenta poderosa para construção de aplicações robustas em JavaScript com segurança adicional.

Utilize como suporte o material “Aula1 - Introdução à linguagem TypeScript” disponível em:

<https://github.com/arleysouza/2dsm/tree/master/T%C3%A9cnicas%20de%20programa%C3%A7%C3%A3o%20I>

Etapas para criação do projeto:

- i. Crie uma pasta chamada `server` (ou nome equivalente) em um local apropriado do seu computador;
- ii. Abra essa pasta no Visual Studio Code;
- iii. No terminal do VS Code, execute o seguinte comando para inicializar um projeto Node.js:

```
npm init -y
```
- iv. Em seguida, instale os pacotes necessários para trabalhar com TypeScript e executar os arquivos diretamente:

```
npm i -D typescript ts-node
```
- v. Crie um arquivo de configuração do TypeScript:

```
npx tsc --init
```
- vi. Edite o arquivo `tsconfig.json` para conter a seguinte configuração mínima, ou seja, substitua o conteúdo do arquivo pelo seguinte JSON:

```
{
  "compilerOptions": {
    "target": "es2020",
    "module": "commonjs",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  }
}
```

- vii. Crie a pasta `src` e dentro dela os arquivos que serão utilizados nesta atividade;
- viii. Crie o arquivo `src/exemplo.js` e coloque nele o seguinte código:

```
function saudacao(nome) {
  return `Olá, ${nome}!`;
}

function media(a, b, c) {
  return (a + b + c) / 3;
}

function listarNomes(nomes) {
  for(let i = 0; i < nomes.length; i++) {
    console.log(saudacao(nomes[i]));
  }
}

const nomes = ["Ana", "Carlos", "Beatriz"];
listarNomes(nomes);

const resultado = media(12, 15, 30);
console.log(`Média: ${resultado}`);
```

- ix. Crie o arquivo `src/exemplo.ts` e coloque nele o seguinte código.

Reescrita em TypeScript com tipagem estática:

```
function saudacao(nome: string): string {
    return `Olá, ${nome}!`;
}

function media(a: number, b: number, c: number): number {
    return (a + b + c) / 3;
}

function listarNomes(nomes: string[]): void {
    for(let i = 0; i < nomes.length; i++) {
        console.log(saudacao(nomes[i]));
    }
}

const nomes: string[] = ["Ana", "Carlos", "Beatriz"];
listarNomes(nomes);

const resultado: number = media(12, 15, 30);
console.log(`Média: ${resultado}`);
```

- x. Atualize o arquivo `package.json` com os seguintes scripts:

```
{
  "name": "server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "exemplo-js": "node ./src/exemplo.js",
    "exemplo-ts": "npx ts-node ./src/exemplo.ts"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "devDependencies": {
    "ts-node": "^10.9.2",
    "typescript": "^5.8.3"
  }
}
```

O script `exemplo-js` executa a versão em JavaScript puro. O `exemplo-ts` executa diretamente o arquivo TypeScript com `ts-node`.

Use `npx` antes de `ts-node` e `nodemon` para evitar problemas se não estiverem instalados globalmente.

- xi. Execução dos arquivos.

Para executar o código em JavaScript:

```
npm run exemplo-js
```

Para executar o código em TypeScript com ts-node:

```
npm run exemplo-ts
```

Exercícios

Veja os vídeos se tiver dúvidas nos exercícios:

Introdução - <https://youtu.be/eGqd9Y0W6-w>

Exercícios 1 a 4 - <https://youtu.be/OzlanU0gb8A>

Exercícios 5 a 8 - <https://youtu.be/IUAvZEmqaXY>

Exercício 1 - Trabalhando com Tipos de União e Detecção de Tipos.

A função `processarValor` aceita diferentes tipos de dados (`number` ou `string`) e retorna um resultado com base no tipo do valor fornecido. Reescreva o código a seguir adicionando as **anotações de tipo necessárias**, utilizando **Union Types** e o operador `typeof`.

```
// arquivo: src/exercicio1.js

function processarValor(valor) {
  if (typeof valor === "number") {
    return valor * 2;
  } else {
    return valor.toUpperCase();
  }
}

const entrada1 = 10;
const entrada2 = "typescript";

console.log("Resultado 1:", processarValor(entrada1));
console.log("Resultado 2:", processarValor(entrada2));
```

Tarefa:

1. Criar o arquivo `src/exercicio1.ts`;
2. Adicionar anotações de tipo para:
 - Parâmetro da função `processarValor`;
 - Tipo de retorno da função `processarValor`;
 - Variáveis `entrada1` e `entrada2`;
3. Validar o comportamento usando `typeof` para lidar com múltiplos tipos corretamente;
4. Adicionar o script no `package.json`:

```
"um-js": "node ./src/exercicio1.js",
"um-ts": "npx ts-node ./src/exercicio1.ts"
```

Exercício 2 - Função Genérica para exibir Arrays de Diferentes Tipos.

Reescreva a função `exibirElementos` a seguir para torná-la genérica, ou seja, capaz de receber um array de **qualquer tipo**, preservando os tipos dos elementos e imprimindo-os um por um.

```
// arquivo: src/exercicio2.js

function exibirElementos(lista) {
  for (let i = 0; i < lista.length; i++) {
    console.log(`Elemento ${i}:`, lista[i]);
  }
}

const numeros = [10, 20, 30];
const palavras = ["um", "dois", "três"];

exibirElementos(numeros);
exibirElementos(palavras);
```

Tarefa:

1. Criar o arquivo `src/exercicio2.ts`;
2. Refatorar a função para usar generics com a notação `<T>`;
3. Garantir que a função aceite qualquer tipo de array mantendo a tipagem segura;
4. Adicionar o seguinte script ao `package.json`:

```
"dois-js": "node ./src/exercicio2.js",
"dois-ts": "npx ts-node ./src/exercicio2.ts"
```

Exercício 3 - Uso de parâmetros opcionais em funções.

A função `multiplicarOuSomar` realiza a multiplicação de dois números se ambos forem passados. Caso apenas um seja informado, ela retorna o número somado a ele mesmo. Reescreva a função com **tipagem explícita** e utilizando **parâmetro opcional** (`?`).

```
// arquivo: src/exercicio3.js

function multiplicarOuSomar(a, b) {
  if (b === undefined) {
    return a + a;
  } else {
    return a * b;
  }
}

console.log(multiplicarOuSomar(5)); // 10
console.log(multiplicarOuSomar(5, 3)); // 15
```

Tarefa:

1. Criar o arquivo `src/exercicio3.ts`;
2. Adicionar a anotação de tipo para:
 - Parâmetro `a` (obrigatório);
 - Parâmetro `b` (opcional);
 - Tipo de retorno da função.
3. Executar o código, observando a mudança de comportamento com um ou dois argumentos;
4. Adicionar ao `package.json`:

```
"tres-js": "node ./src/exercicio3.js",  
"tres-ts": "npx ts-node ./src/exercicio3.ts"
```

Exercício 4 - Manipulação e transformação de arrays tipados.

A função `duplicarValores` recebe um array de números e retorna um **novo array** com os valores **dobrados**. Reescreva a função com a **tipagem correta** de entrada e saída, utilizando estruturas já conhecidas (sem `map`, apenas for tradicional).

```
// arquivo: src/exercicio4.js  
  
function duplicarValores(lista) {  
  const resultado = [];  
  
  for (let i = 0; i < lista.length; i++) {  
    resultado.push(lista[i] * 2);  
  }  
  
  return resultado;  
}  
  
const valores = [2, 4, 6];  
const duplicados = duplicarValores(valores);  
  
console.log("Original:", valores);  
console.log("Dobrados:", duplicados);
```

Tarefa:

1. Criar o arquivo `src/exercicio4.ts`;
2. Adicionar a tipagem explícita:
 - `lista` como `number[]`;
 - `resultado` como `number[]`;
 - Tipo de retorno da função como `number[]`;
3. Executar o código para validar o funcionamento;
4. Adicionar ao `package.json`:

```
"quatro-js": "node ./src/exercicio4.js",  
"quatro-ts": "npx ts-node ./src/exercicio4.ts"
```

Exercício 5 - Trabalhando com arrays de objetos e interface.

A função aprovados receberá uma lista de objetos representando estudantes, cada um com nome e nota, e retorna somente os estudantes com nota maior ou igual a 7. Crie uma **interface** para tipar os objetos.

```
// arquivo: src/exercicio5.js  
  
const estudantes = [  
  { nome: "Ana", nota: 8 },  
  { nome: "Carlos", nota: 6 },  
  { nome: "Beatriz", nota: 9 },  
  { nome: "Eduardo", nota: 5 }  
];  
  
// função que retorna os estudantes com nota >= 7  
function aprovados(lista) {  
  const resultado = [];  
  
  for (let i = 0; i < lista.length; i++) {  
    if (lista[i].nota >= 7) {  
      resultado.push(lista[i]);  
    }  
  }  
  
  return resultado;  
}  
  
console.log("Aprovados:", aprovados(estudantes));
```

Tarefa:

1. Criar o arquivo src/exercicio5.ts;
2. Criar uma **interface chamada Estudante** com os campos:
 - o nome: string
 - o nota: number
3. Tipar:
 - o O array estudantes como Estudante[];
 - o O parâmetro da função aprovados como Estudante[];
 - o O retorno como Estudante[].
4. Adicionar o script ao package.json:

```
"cinco-js": "node ./src/exercicio5.js",  
"cinco-ts": "npx ts-node ./src/exercicio5.ts"
```

Exercício 6 - Interfaces compostas e manipulação de dados aninhados.

Crie as interfaces Endereco e Pessoa, na qual o endereço será um **objeto interno**. A função listarCidades irá listar os dados das pessoas com nome e cidade.

```
// arquivo: src/exercicio6.js

const pessoas = [
  {
    nome: "João",
    idade: 30,
    endereco: {
      cidade: "São Paulo",
      estado: "SP"
    }
  },
  {
    nome: "Marina",
    idade: 25,
    endereco: {
      cidade: "Campinas",
      estado: "SP"
    }
  }
];

// Função para listar nomes e cidades
function listarCidades(pessoas) {
  for (let i = 0; i < pessoas.length; i++) {
    console.log(`${pessoas[i].nome} mora em ${pessoas[i].endereco.cidade}`);
  }
}

listarCidades(pessoas);
```

Tarefa:

1. Criar o arquivo src/exercicio6.ts;
2. Criar duas interfaces:
 - Endereco com as propriedades:
 - cidade: string
 - estado: string
 - Pessoa com as propriedades:
 - nome: string
 - idade: number
 - endereco: Endereco

3. Tipar:

- O array pessoas como Pessoa[];
- O parâmetro da função listarCidades como Pessoa[];

4. Adicionar o script ao package.json:

```
"seis-js": "node ./src/exercicio6.js",  
"seis-ts": "npx ts-node ./src/exercicio6.ts"
```

Exercício 7 - União de tipos dentro de interfaces.

Crie uma interface Contato com os dados de uma pessoa. A propriedade telefone pode ser informada como **número (number)** ou **texto (string)**, e a função exibirContatos deve mostrar a lista de contatos com seus nomes e formas de contato.

```
// arquivo: src/exercicio7.js  
  
const contatos = [  
  { nome: "Lucia", telefone: "11987654321" },  
  { nome: "Marcos", telefone: 11981234567 }  
];  
  
// Função que exibe nome e telefone  
function exibirContatos(lista) {  
  for (let i = 0; i < lista.length; i++) {  
    console.log(`Nome: ${lista[i].nome} - Telefone: ${lista[i].telefone}`);  
  }  
}  
  
exibirContatos(contatos);
```

Tarefa:

1. Criar o arquivo src/exercicio7.ts;
2. Criar a interface Contato com as propriedades:
 - nome: string
 - telefone: string | number
3. Tipar:
 - O array contatos como Contato[];
 - A função exibirContatos para receber Contato[];
4. Executar o código e verificar que aceita telefones com ou sem aspas;
5. Adicionar ao package.json:

```
"sete-js": "node ./src/exercicio7.js",  
"sete-ts": "npx ts-node ./src/exercicio7.ts"
```

Exercício 8 - Uso de typeof para refinamento de tipos em interfaces.

Crie uma interface Produto que possui uma propriedade preco, a qual pode ser do tipo number ou string. A função exibirPrecoFormatado deverá:

- Verificar o tipo do preco com typeof;
- Se for string, exibir com a mensagem "valor informado em texto";
- Se for number, formatar o número como valor monetário (R\$...).

Código-base inicial (incompleto):

```
// arquivo: src/exercicio8.ts

const produtos = [
  { nome: "Notebook", preco: 2500 },
  { nome: "Monitor", preco: "1400" },
  { nome: "Teclado", preco: 230 }
];

function exibirPrecoFormatado(lista) {
  for (let i = 0; i < lista.length; i++) {
    // verificar o tipo de preco e mostrar de forma diferente
  }
}

exibirPrecoFormatado(produtos);
```

Tarefa:

1. Criar o arquivo src/exercicio8.ts;
2. Criar a interface Produto com:
 - nome: string
 - preco: number | string
3. Tipar o array produtos e a função exibirPrecoFormatado;
4. Dentro da função, usar typeof para aplicar um **refinamento de tipo** e exibir o preço corretamente;
5. Adicionar o script ao package.json:

```
"oito-ts": "npx ts-node ./src/exercicio8.ts"
```

Saída esperada:

Notebook - Preço: R\$ 2500.00

Monitor - Preço: valor informado em texto (1400)

Teclado - Preço: R\$ 230.00